

Python Code

```
1  import heapq
2  import math
3
4
5  def calcular_heuristica(atual, destino):
6      return math.sqrt((atual[0] - destino[0])**2 + (atual[1] - destino[1])**2)
7
8  class A_star:
9      #No caso vamos considerar o fitness como uma função que recebe dois args!
10     def __init__(self, mapa, fitness=calcular_heuristica,):
11         self.fit = fitness
12         self.lin = len(mapa)
13         self.cols = len(mapa[0])
14         self.caminho = None
15         self.custo = 0
16         inicio = None
17         fim = None
18         #Procuramos o inicio e o fim!
19         for i in range(self.lin):
20             for j in range(self.cols):
21                 if mapa[i][j] == 'I':
22                     inicio = (i, j)
23                 elif mapa[i][j] == 'F':
24                     fim = (i, j)
25
26         if inicio == None or fim == None:
27             print("Error! Inicio ou fim não encontrados!")
28             return
29         lista_aberta = []
30         contador = 0
31         # Inicialmente o custo em 'I' é 0. E o caminho percorrido tbm é 'I'
32         heapq.heappush(lista_aberta, (0, contador, 0, inicio, [inicio]))
33         # Dicionário para encontrar o menor G de cada célula, G É CUSTO NO
CASO!
34         melhor_g = { inicio: 0 }
35         celulas_expandidas = set() # lista de células expandidas
36         direcoes = [(0, 1), (1, 0), (0, -1), (-1, 0)] # Movimentos
possíveis: Direita, Baixo, Esquerda, Cima
37         while lista_aberta:
38             f, _, g_atual, atual, caminho = heapq.heappop(lista_aberta)
39             #Pega a célula com menor custo F
40             if atual in celulas_expandidas:
41                 continue
42             celulas_expandidas.add(atual)
43             if atual == fim: #CHEGAMOS NO FIM!
44                 self.caminho = caminho
45                 self.custo = g_atual
46             for dr, dc in direcoes: # Explora as direções distintas
47                 vizinho_r, vizinho_c = atual[0] + dr, atual[1] + dc
48                 vizinho = (vizinho_r, vizinho_c)
```

```
48         if 0 <= vizinho_r < self.lin and 0 <= vizinho_c < self.cols:
49             valor_celula = mapa[vizinho_r][vizinho_c]
50             #caminho bloqueado!
51             if valor_celula == '*':
52                 continue
53             custo_passo = 0
54             if valor_celula != 'F' and valor_celula != 'I':
55                 custo_passo = int(valor_celula)
56
57             # Calcula o novo custo acumulado (G)
58             novo_g = g_atual + custo_passo
59             if vizinho not in melhor_g or novo_g < melhor_g[vizinho]:
60                 melhor_g[vizinho] = novo_g
61                 h = fitness(vizinho, fim)
62                 novo_f = novo_g + h
63                 novo_caminho = caminho + [vizinho]
64                 contador += 1
65                 heapq.heappush(lista_aberta, (novo_f, contador, novo_g,
vizinho, novo_caminho))
```